

# 第七章 深度Q网络进阶技巧

本章介绍一些改进 DQN 的技巧。

## 双深度Q网络 DDQN

问题：基础的 DQN 中，目标为

$$y_t = r_t + \gamma(1 - d_t) \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a')$$

实际应用中，我们发现这样做会大大**高估 Q 的真实值**，原因是在计算目标时，**选择动作**（根据 Q 值大小选动作）和**评价动作**（采用该动作得到评价 Q）是**同一个网络**，也就是目标网络  $Q_{\bar{\theta}}$ 。由于这里的 Q 本身就有误差，如果每次都选择并执行最大的 Q，会导致目标不断偏离实际向上攀升，导致高估。（换句话讲，虽然整体来看多次采样得到的期望是准确的，但是**单次误差 + 每次取最大值就会导致目标持续上升。**）

解决方法：将**选择和评价分离**：**用在线网络选择动作，目标网络评价动作**，即先用在线网络选择动作：

$$a^* = \arg \max_{a'} Q_{\theta}(s_{t+1}, a')$$

然后再用目标网络评价这个动作：

$$y_t^{\text{DDQN}} = r_t + \gamma(1 - d_t) Q_{\bar{\theta}}(s_{t+1}, a^*)$$

对比如下：

| 方法     | 谁选择动作                   | 谁评价动作                   |
|--------|-------------------------|-------------------------|
| 基础 DQN | 目标网络 $Q_{\bar{\theta}}$ | 目标网络 $Q_{\bar{\theta}}$ |
| DDQN   | 在线网络 $Q_{\theta}$       | 目标网络 $Q_{\bar{\theta}}$ |

举个例子：假如当前有在线网络输出  $Q_{\theta}(s', \cdot) = [4, 7, 3]$ ，目标网络输出  $Q_{\bar{\theta}}(s', \cdot) = [8, 5, 6]$ 。

基础 DQN：直接看目标网络输出，选最大的 8。这里就有可能 8 是偏大的值，导致目标高估。

DDQN：先看在线网络输出，选第二个，对应到目标网络是 5。

为什么这样做可以减少高估？主要是来自**误差的不匹配**：不管用哪个网络来估计 Q 都会有误差，这一点不可避免。但是我们可以做到**尽量避开出现大误差的值**：基础 DQN 中，只要一个 Q 值的预测出现了很高的高估误差，就**必然会选择这个高估的 Q 值**计算目标；而 DDQN 中两个网

络的 Q 值预测是分离的，即使在线网络选择了被高估的动作，目标网络评价时该动作仍被高估的概率大大减小，从而抑制高估。

(DQN：一旦**出现高估，极大概率被采纳**；DDQN：只有两个网络都在同一动作出现高估才会导致目标高估，**任意一方估计值正常都能够避免高估**。)

**拓展**：DQN 高估的问题在 Q-learning 中也存在，只不过不是很严重。与之对应的 Q-learning 也有改进技巧：Double Q-learning，通过维护两张 Q 表来实现。

## 竞争深度Q网络 Dueling DQN

---

问题：Q 实际上是由两个因素决定的：**当前状态如何 + 选择动作如何**。

举个例子：被敌人打到残血（坏状态），无论选择多好的战略都无济于事；反之如果已经胜券在握（好状态），怎么打都能赢。这两种就是典型的状态 > 动作。

但当敌我势均力敌时，策略的选择（动作的好坏）就尤为重要。

传统的 DQN 只学习了 Q，而不知道这个 Q 到底是主要由状态决定还是动作决定的。此外，传统的 DQN 在面对同一状态、多个动作的情况下，只采样其中一个动作来学习，对于其他动作的估计帮助有限。

解决方法：**把 Q 分成两部分：状态价值和动作优势**，即

$$Q(s, a) = V(s) + A(s, a)$$

其中各动作的 A 的平均值设定为 0，表示“该动作的收益相较于平均值如何”。

这样做首先能够**优化部分采样对整体的帮助**：比如某次训练提升了 V 值，那么这个状态下所有动作的 Q 都会跟着提升，因为 **V 是共享的**。带来了效率的提升。即使某些动作很少被采样，状态价值信息也可以帮助估计它们；当“当前状态好不好”比“具体选哪个动作”更重要时，学习会更高效。

总结起来，Dueling DQN 做的事情可以概括为：直接学习每个  $Q(s, a) \implies$  先学习  $V(s)$ ，再学习  $A(s, a)$ ，最后合成  $Q(s, a)$ 。

它的核心价值是：**把“这个状态好不好”和“在这个状态下哪个动作更好”分开学习，从而更充分地利用状态层面的信息，提高 Q 值估计的数据效率**。

## 优先级经验回放 PER

---

问题：我们已经采集的经验里面学习的程度各有不同：有的已经拟合得很好了，有的仍然存在较大误差。若随机取样，就像没有针对性地复习错题本中的题目一样，效率低下。

解决方法：根据训练的程度如何**给经验分配优先级**，高优先级的经验有**更大的概率被抽中参与训练**。

一些技术细节：

- **TD 误差**：对于某一条经验，目标值与当前估计值之间的差的绝对值。TD 误差越大，越说明学习没到位，需要更高的优先级。
- **重要性采样权重**：PER 会改变从经验集里采样的分布，所以要为每个经验加上权重来修正损失：高优先级的经验其权重较小，反之亦然。

好处：PER 能**更集中训练误差大的样本**，提升模型表现。但也有过度关注噪声样本的风险。

## 多步奖励

本质上是在**蒙特卡洛和时序差分之间找平衡**。

可以将原始 DQN 的一步时序差分拓展到  $n$  步。此时的目标为

$$y_t^{(n)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n (1 - d) \max_{a'} Q_{\theta}(s_{t+n}, a')$$

区别如下：

| 方法    | 使用的回报                | 优点                  | 缺点            |
|-------|----------------------|---------------------|---------------|
| 一步 TD | 1 个真实奖励 + 1 次 Q 估计   | 更新快、方差较小            | 依赖 Q 估计，奖励传播慢 |
| 蒙特卡洛  | 整个回合的真实回报            | bootstrap 偏差小、奖励传播快 | 等待时间长、方差较大    |
| 多步方法  | 多个真实奖励 + 近处 1 次 Q 估计 | 兼顾两者                | 需要选择步数 $n$    |

- $n$  较小：bootstrap 次数多，目标方差较小，但可能有较大估计偏差，奖励传播慢；
- $n$  较大：真实奖励更多，奖励传播快，对当前 Q 网络依赖较小，但累积奖励的随机性更强，方差更大；
- $n$  取中间值：在偏差和方差、传播速度和稳定性之间折中。

注意： $n$  的变化会**影响经验构造的流程**。

普通 DQN 一步交互后就可以保存  $(s_t, a_t, r_t, s_{t+1})$ ；而多步 DQN 则需要暂时保存一小段连续轨迹。

下面是一个例子：假设  $n = 3$ ，智能体经历：

$$(s_t, a_t, r_t), (s_{t+1}, a_{t+1}, r_{t+1}), (s_{t+2}, a_{t+2}, r_{t+2}), s_{t+3}$$

然后构造一条三步经验：

$$(s_t, a_t, r_t + \gamma r_{t+1} + \gamma^2 r_{t+2}, s_{t+3})$$

再把它放入经验回放池。

也就是：连续交互  $n$  步  $\rightarrow$  累积  $n$  步奖励  $\rightarrow$  构造  $n$ -step 经验  $\rightarrow$  存入回放池。

## 噪声网络 NoisyNet

问题：之前我们学习了  $\epsilon$ -贪心和玻耳兹曼探索两种探索的方法，它们的共同点在于**只关注当前这一次动作的选择**，而没有考虑多个动作连在一起构成的完整动作链。事实上，如果任务需要连续采取一组正确动作才能获得奖励，那么每一步独立随机可能很难发现完整的成功策略。

解决方法：**不在每一次做决定时动手脚，而是直接改策略**：每回合开始时，给当前网络  $\theta$  的每个参数加上一个随机的、**本回合固定**的高斯噪声，得到**当前回合使用的网络  $\tilde{\theta}$** 。也就是说，**先改变整个 Q 函数，再根据这个新的 Q 函数做贪心决策**。探索行为由状态和当前网络参数共同决定，而不是每一步独立抛硬币。这比每一步独立随机更像是在尝试不同的“策略方案”，所以有助于深度探索。

NoisyNet 能够**形成更加连贯、依赖状态的探索行为**，尤其适合需要连续尝试一整套动作才能获得奖励的任务。

## 分布式Q函数 Distributed Q-function

问题：普通的 Q 函数只保留了奖励的期望，而没有其分布、方差等**更详细的信息**。

解决方法：使用分布式 Q 函数，学习  $Z(s, a)$ （随机变量）而非  $Q(s, a)$ （期望）。

有

$$Q(s, a) = \mathbb{E}[Z(s, a)]$$

表示方法：Z 划分为几个区间，例如  $[0, 10]$ 、 $[10, 20]$ 、 $[20, 30]$  之类的，输出的是每个区间的概率。

例如：

$$\begin{aligned} p(s, a_1) &= [0.1, 0.2, 0.4, 0.2, 0.1] \\ p(s, a_2) &= [0.5, 0.1, 0.1, 0.1, 0.2] \\ p(s, a_3) &= [0.05, 0.05, 0.1, 0.3, 0.5] \end{aligned}$$

如果区间能覆盖所有可能取值，则每一个  $p$  的值之和应当为 1。

得到  $Z$  后，对应的  $Q$  可以通过加权求和得到。

总而言之，分布式  $Q$  函数学习的是“未来可能得到哪些回报，以及每种回报出现的概率”；它保留了均值之外的风险和不确定性信息。

## Rainbow彩虹

把上面的这些技巧组合起来使用，总结如下表：

| 方法                         | 详情                                                                                                                                                                                                                                                                                                        |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 双深度 Q 网络 (DDQN)            | <b>改进环节：</b> TD 目标计算； <b>核心改动：</b> 在线网络选择下一动作 $a^* = \arg \max_{a'} Q_{\theta}(s', a')$ ，目标网络评价该动作 $y = r + \gamma Q_{\bar{\theta}}(s', a^*)$ ； <b>主要效果：</b> 将动作选择与价值评价分开，减少最大化操作造成的 $Q$ 值高估，目标值通常更可靠； <b>代价与注意：</b> 只能减轻高估，不能保证完全消除；在线网络与目标网络仍然相关。                                                       |
| 竞争深度 Q 网络 (Dueling DQN)    | <b>改进环节：</b> $Q$ 网络结构； <b>核心改动：</b> 共享特征后分成状态价值分支 $V(s)$ 和动作优势分支 $A(s, a)$ ，再合成 $Q(s, a) = V(s) + A(s, a) - \text{mean}_{a'} A(s, a')$ ； <b>主要效果：</b> 将“状态整体好不好”和“某动作相对好不好”分开学习；一次经验对状态价值的更新可以帮助多个动作，提高数据利用率，尤其适合许多动作价值相近的状态； <b>代价与注意：</b> 只改变网络结构，不改变 TD 目标；并非所有任务都明显受益；减去平均优势是为了消除 $V$ 与 $A$ 分解不唯一的问题。 |
| 优先级经验回放 (PER)              | <b>改进环节：</b> 回放池采样方式； <b>核心改动：</b> 计算 TD 误差 $\delta_i = y_i - Q_{\theta}(s_i, a_i)$ ，以 $p_i = \text{abs}(\delta_i) + \epsilon_p$ 设置优先级，TD 误差大的经验获得更高采样概率； <b>主要效果：</b> 集中训练当前预测错误较大的经验，减少在已学好样本上的重复更新，提高样本利用效率； <b>代价与注意：</b> 非均匀采样会引入偏差，需要重要性采样权重修正；也可能过度关注噪声、异常奖励或过时的高误差经验。                             |
| 多步方法 (Multi-step Learning) | <b>改进环节：</b> TD 目标的时间跨度； <b>核心改动：</b> 使用连续 $n$ 步真实奖励，再在第 $n$ 步进行 bootstrap: $y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, \cdot)$ ； <b>主要效果：</b> 让延迟奖励更快传播到前面的状态，并减少目标对当前 $Q$ 估计的依赖；在一步 TD 与蒙特卡洛之间取得平衡； <b>代价与注意：</b> $n$ 较小则 bootstrap 偏差较明显， $n$ 较大则回报方差、存储复杂度以及对行为轨迹的依赖通常增加。    |

| 方法                          | 详情                                                                                                                                                                                                                                                                                                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 噪声网络<br>(NoisyNet)          | <b>改进环节</b> ：探索机制； <b>核心改动</b> ：在网络参数中加入噪声，例如 $\tilde{\theta} = \mu + \sigma \odot \xi$ ，再采用贪心动作 $a = \arg \max_a Q_{\tilde{\theta}}(s, a)$ ；课本中一个回合内保持同一组噪声； <b>主要效果</b> ：从每一步随机动作转变为对整套策略进行扰动，形成更连贯、依赖状态的探索，更有利于发现需要连续多步行动才能获得的奖励； <b>代价与注意</b> ：实现和调参比 $\epsilon$ -greedy 复杂；噪声过大会破坏已有策略，过小则探索不足；不同实现的噪声重采样频率可能不同。 |
| 分布式 Q 函数<br>(Distributed Q) | <b>改进环节</b> ：价值表示和训练目标； <b>核心改动</b> ：学习累计回报随机变量 $Z(s, a)$ 的概率分布，而不只学习期望；普通 Q 值可由 $Q(s, a) = \mathbb{E}[Z(s, a)]$ 得到； <b>主要效果</b> ：保留平均值之外的波动、风险和多峰信息，提供更丰富的训练信号，也能支持风险敏感决策； <b>代价与注意</b> ：学习的是回报分布，不是动作概率；网络输出和损失更复杂；离散分布方法还需设定回报范围并投影，范围过窄会裁剪极端回报。                                                                   |